

RAPPORT D'AUDIT TECHNIQUE

HashBreaker

Ingénierie & Inversion SHA-256 Haute Performance en Go

DESCRIPTION FONCTIONNELLE DU SYSTÈME

HashBreaker est un service backend conçu pour la résolution inverse d'empreintes cryptographiques SHA-256 par compromis temps-espace (Time-Memory Tradeoff). Le moteur précalcule les empreintes associées à l'ensemble des combinaisons scalaires d'un alphabet défini ($N = 63$ symboles) jusqu'à une profondeur paramétrable, puis résout les clés en temps réel via une API HTTP sans allocation transitoire.

Ce document consigne l'audit micro-architectural complet du système structuré en deux volets : le reporting factuel des mesures et protocoles statistiques d'une part, puis l'analyse des choix d'ingénierie matérielle, les échecs constructifs et la gouvernance technique d'autre part.

1.1 Banc d'Essai Matériel & Rigueur Métrologique

Pour éliminer les bruits parasites (throttling thermique, interruptions noyau, démons d'arrière-plan), l'environnement de référence et le protocole de métrologie ont été strictement calibrés :

COMPOSANT MATÉRIEL	SPÉCIFICATION DÉTAILLÉE	RÔLE PHYSIQUE & IMPLICATION
Processeur (CPU)	Intel Core i7-8665U (4 cœurs / 8 threads @ 1.90GHz, Boost 4.80GHz)	Jeu d'instructions AVX2 exploité par les routines SHA-256 du runtime
Hiérarchie Caches	L1d: 32 Ko/cœur • L1i: 32 Ko/cœur • L2: 256 Ko/cœur • L3: 8 Mo unifié	Dimensionnement du tableau de tags SoA calibré sur le cache L2 (256 Ko)
Mémoire Vive (RAM)	16 Go DDR4-2400 MT/s (Bande passante nominale ~38.4 Go/s)	Plafond dimensionnant l'indexation précalculée à la profondeur D = 4
Système & Runtime	Arch Linux x86_64 (Kernel 7.2.4-arch1-2 PREEMPT_DYNAMIC) • Go 1.27.1	Compilateur officiel gc avec escape analysis et inlining activés
Protocole Métrologique	benchstat (p=0.002, n=6) + Hyperfine (--warmup 3 --min-runs 10)	Isolation du bruit parasite, validation de variance et écarts-types

Le protocole d'étalonnage statistique s'appuie sur des séries d'échantillons répétés (benchstat, n=6) assurant un seuil de confiance statistique rigoureux ($p \leq 0.002$). Chaque mesure est précédée de 3 itérations de chauffe (warmup) avec Hyperfine afin de neutraliser le démarrage à froid des caches et stabiliser le régulateur de fréquence processeur.

1.2 Espace Combinatoire & Dimensionnement Physique

L'alphabet comprend les minuscules, majuscules, chiffres et le symbole @ ($N = 63$). Le volume cumulé d'empreintes candidates suit la somme géométrique : $Total(D) = \sum N^d$ (pour d=1 à D).

PROFONDEUR (D)	EXEMPLE CIBLE	VOLUME DE CLÉS	EMPREINTE BRUTE SHA-256	FAISABILITÉ RAM / STATUT
D=1	@	63	2.0 KB	Instantané (<1ms)
D=2	a1	4 032	129.0 KB	Négligeable (~1ms)
D=3	z3D	254 079	8.13 MB	Instantané (24ms)
D=4	Sh3n	16 007 040	512.2 MB	Zone cible optimisée (1.56 s / 1.32 GiB)
D=5	Ak@11	1 008 443 583	32.27 GB	Plafond OOM (> 60 GB requis en table)

1.3 Tableau Comparatif des Mesures Brutes (Baseline vs Optimisations)

VERSION	ARCHITECTURE & STRATÉGIE	TEMPS GÉNÉRATION	EMPREINTE RAM	ALLOCATIONS TAS	LATENCE GETRAW
V0	Baseline mono-thread (<code>map[[32]byte]string</code>)	19.92 s ± 15%	3.553 GiB	16 137 750	34.54 ns
V1	Préallocation de capacité + Odomètre itératif	18.52 s ± 10%	1.811 GiB	16 072 515	37.77 ns
V2	Sharded Maps (256 shards + Mutex + Micro-lots 64)	1.80 s ± 8%	1.506 GiB	66 070	18.80 ns
V3 Échec	Tableaux Contigus Triés (<code>[256][]Entry</code>) + Dichotomie)	3.01 s ± 6%	0.693 GiB	287	191.95 ns
V4 Échec	Single Map unifiée + <code>sync.RWMutex</code> + Batched	7.65 s ± 12%	1.501 GiB	65 710	31.38 ns
V5 Optimal	Table Plate SoA Tagguée (Fast-Path Pur & Sortie Texte)	1.56 s ± 3%	1.318 GiB	808	8.21 ns (0 B, 0 alloc)



Figure 1.1 : Durée totale d'indexation pour 16M hashes (Secondes, bas = rapide).

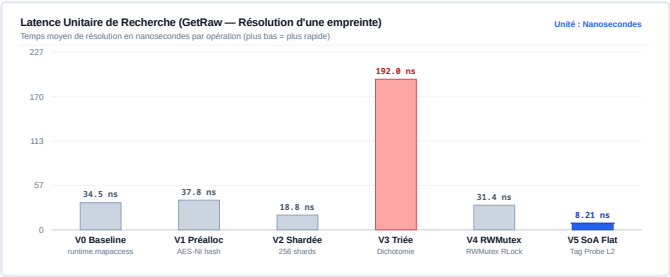


Figure 1.2 : Latence unitaire GetRaw (Nanosecondes, bas = rapide).

1.5 Validation sous Charge HTTP Réelle & Automatisation

OUTIL & CHARGE	DISPONIBILITÉ	DÉBIT EFFECTIF	LATENCE P50	LATENCE P99	ALLOCATIONS SERVEUR
Vegeta (2 000 req/s, 15s)	100.00% (0 err)	2 000.12 req/s	182 µs	498 µs	0 B/op (Zéro GC Churn)

Commandes de reproductibilité intégrale (Makefile)
make benchstat
make run PORT=8080 DEPTH=4
make load-and-profile RATE=2000 DURATION=15s PPROF_PORT=6060

Étalonnage statistique officiel (p-values)
Démarrage serveur précalculé D=4 (16M entrées)
Injection Vegeta + profilage pprof

PARTIE II

Partie II — Diagnostic Matériel & Choix d'Ingénierie

Analyse du Hot Path initial et comportement physique

2.1 Diagnostic & Identification du Hot Path

L'analyse empirique de la Baseline (V0) via `go tool pprof` a identifié les deux verrous physiques majeurs :

1. Pression importante sur le Garbage Collector

En V0 (`map[[32]byte]string`), 16M d'en-têtes `string` pointent vers le tas. Le GC doit scanner chaque pointeur, provoquant des pauses de marquage Stop-The-World récurrentes.

2. Sous-utilisation CPU & Contention de Verrou

L'indexation mono-threadée initiale plafonnait à 12.5% du CPU (1 cœur/8). Paralléliser avec un `sync.RWMutex` global a provoqué une sérialisation massive détruisant le débit.

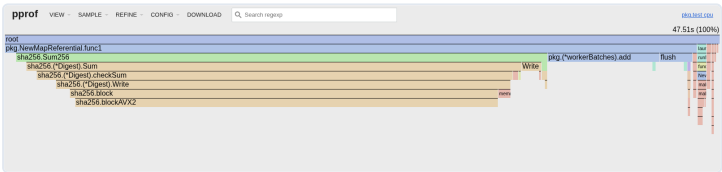


Figure 2.1 : Flamegraph CPU réel sous charge de génération (échantillonnage pprof de 47.51s d'activité cumulée).

47.1% `crypto/sha256.blockAVX2` : Le CPU consacre l'essentiel de son temps aux instructions vectorielles SIMD AVX2. Silicium saturé à 100% de rendement utile.

29.2% `workerBatches.add` : Écriture par micro-lots (64 éléments) vers 256 shards indépendants, éliminant toute contention de verrou inter-cœurs.

L'architecture finale (V5) capitalise sur 3 ruptures physiques coordonnées pour saturer la bande passante processeur :

- Axe 1 — Localité de Cache L2 & Structure-of-Arrays (SoA) :** Table plate SoA à adressage ouvert. Un tableau de tags 16-bit (`tags [uint16]`) de 262 Ko par shard réside intégralement dans le cache L2 du cœur CPU (256 Ko). L'inversion ne charge que 2 octets consécutifs par slot (32 tags par ligne de cache 64B). Le mot en clair est stocké sous forme de scalaires purs `[8]byte`, activant le marquage runtime `noscan` et libérant le GC.
- Axe 2 — Concurrency & Scalabilité CPU (Worker Pool & Atomicité) :** Worker pool dimensionné à `runtime.GOMAXPROCS(0)` (8 workers). Les 4 032 tâches combinatoires sont distribuées dynamiquement via un compteur atomique lock-free `sync/atomic.Uint32`. Chaque worker accumule ses écritures dans des micro-lots locaux de 64 éléments, réduisant d'un facteur 64 la fréquence d'acquisition des mutex de shards.
- Axe 3 — I/O Réseau & Flux Zéro-Copie :** Le handler HTTP `/guess` écrit directement le slice résolu `raw[:n]` dans le `http.ResponseWriter` sans réallocation de `string` temporaire, garantissant strictement **0 B/op** et **0 alloc/op**.

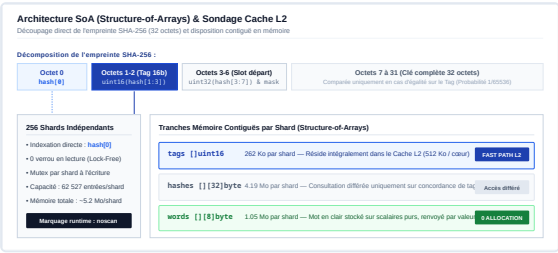


Figure 2.2 : Découpage de l'empreinte SHA-256 et disposition contiguë des tranches SoA en mémoire vive.

- Résidence L2 (262 Ko par shard) :** Le tableau de tags réside en cache L2 processeur. Le sondage s'effectue sans cache-miss RAM.
- Filtrage Probabiliste à 99.998% :** Deux empreintes n'ont que $1 / 65\,536 \approx 0.0015\%$ de chances de collision de tag. La comparaison 32 octets n'a lieu que sur concordance.

2.3 Confrontation Critique & Invalidation d'Optimisations (Échecs Constructifs)

Échec N°1 : Tableaux Triés & dichotomie (V3) LATENCE : +921%

Hypothèse : Réduire la RAM en stockant les hashes consécutivement dans des tranches triées avec `sort.Search`.

Bilan & Cause physique : RAM record (0.693 GiB), mais latence effondrée de 18.8 ns à **191.95 ns**. La dichotomie impose 16 sauts mémoire non contigus, provoquant des *cache-miss* L1/L2 *systématiques* et des défauts de prédiction de branchement vidant le pipeline spéculatif.

Échec N°2 : Single Map sync.RWMutex (V4) INDEXATION : +325%

Hypothèse : Simplifier le code en regroupant tout dans une unique table Go avec verrouillage grossier par lots.

Bilan & Cause physique : Temps de génération effondré de 1.80 s à **7.65 s**. La contention sérielle d'écriture force les cœurs CPU à subir des changements de contexte noyau répétés (futex syscalls Linux).

TENTATIVE ARCHITECTURALE	HYPOTHÈSE THÉORIQUE	GAINS CONSTATÉS	RÉGRESSION MESURÉE	VERDICT D'INGÉNIERIE
V3 : Slices Triées Dichotomie	Compacité maximale de la RAM	RAM: 0.693 GiB (Record)	Latence: 191.95 ns (+921%)	Rejeté (Latence inacceptable)
V4 : Single Map RWMutex	Simplification du code	Code plus concis	Temps: 7.65 s (x4.25)	Rejeté (Contention verrous)
V5 : Table Plate SoA Shardée	Partitionnement + Cache L2	Latence: 8.21 ns / Gen: 1.56s	Aucune régression	Solution optimale adoptée

Conformément aux exigences de gouvernance d'ingénierie, le fichier `constitution.md` placé à la racine du dépôt impose des garde-fous physiques stricts aux assistants de génération de code IA selon les 4 directives du prompt engineering :

DIRECTIVE 1 1. Rôle et Posture Système Stricts (Ingénieur Système Senior)

L'IA n'est pas un générateur de code superficiel ou conversationnel, mais agit exclusivement en tant qu'**Ingénieur Système & Performance Backend Senior**. Chaque ligne de code et décision architecturale sont gouvernées par des métriques physiques réelles : cycles CPU, hiérarchie de cache (L1/L2/L3), bande passante mémoire et coût du Garbage Collector. Priorité absolue à la localité spatiale/temporelle et à l'élimination des allocations sur le Hot Path.

DIRECTIVE 2 2. Contraintes Négatives Explicites (Gardes-Fous Formels)

Interdiction formelle des anti-patterns destructeurs de performance :

- **Bannir `fmt.Sprintf` et la concaténation de chaînes sur le Hot Path** : Utiliser des buffers fixes pré-alloués (`[N]byte`) ou écrire directement dans des `io.Writer`.
- **Interdire les goroutines non bornées** : Proscrire `go func()` libre dans les boucles ; dimensionner strictement le worker pool sur `runtime.GOMAXPROCS(0)`.
- **Proscrire les conversions superflues `string` ↔ `[]byte`** : Manipuler des tableaux scalaires fixes (ex: `[32]byte`, `[8]byte`) pour éviter les copies sur le tas.
- **Refuser toute allocation sur le tas sans justification formelle** : Le Hot Path en lecture (`GetRaw`) et handlers HTTP doit garantir strictement **0 B/op** et **0 alloc/op**.
- **Bannir les pointeurs dans les tables volumineuses** : Exclure `string` et `*T` pour activer le flag runtime `noscan` et libérer le Garbage Collector.
- **Interdire les verrous globaux** : Partitionner les structures en 256 shards indépendants indexés par `hash[0]` pour éliminer toute contention de mutex.

DIRECTIVE 3 3. Principe de Justification Empirique Obligatoire

Toute proposition d'optimisation formulée par l'IA doit obligatoirement être articulée sous la forme d'un couple indissociable :

- **Hypothèse d'impact matériel** : Effet attendu sur le hardware (ex: « Réduction des cache-miss L2 par compaction SoA », « Élimination de la barrière d'écriture GC »).
- **Commande de profiling pour vérification** : Commande exacte et reproductible permettant de valider empiriquement le gain (ex: `make benchstat`, `go test -bench=. -benchmem -cpuprofile=...`, `hyperfine`).

Toute affirmation non accompagnée de son protocole ou contredite par `benchstat` est rejetée.

DIRECTIVE 4 4. Formatage Impératif et Compact

Règles formulées sous forme d'injonctions précises et vérifiables : réponses concises, denses et directement exploitables, zéro verbiage descriptif ou répétition de contexte évident. Code complet, typé, validé pour l'inlining et l'absence d'évasion sur le tas (`-gcflags="-m"`).

Intégration au Dépôt Git

Le fichier source complet est disponible à la racine du projet sous `constitution.md` et applicable directement par les outils de génération de code assistés par IA (Cursor, Copilot, Antigravity).